**Primer entorno de desarrollo: terminal, Git y primer commit (Bloque 2) / Inicializa tu repositorio y haz tu primer commit [Actividad práctica]**

✓ **Hecho:** Ver

📘 **UNIDAD U1.2** · Setup técnico: terminal, Git, entorno Python y cliente HTTP

# Inicializa tu repositorio y haz tu primer commit

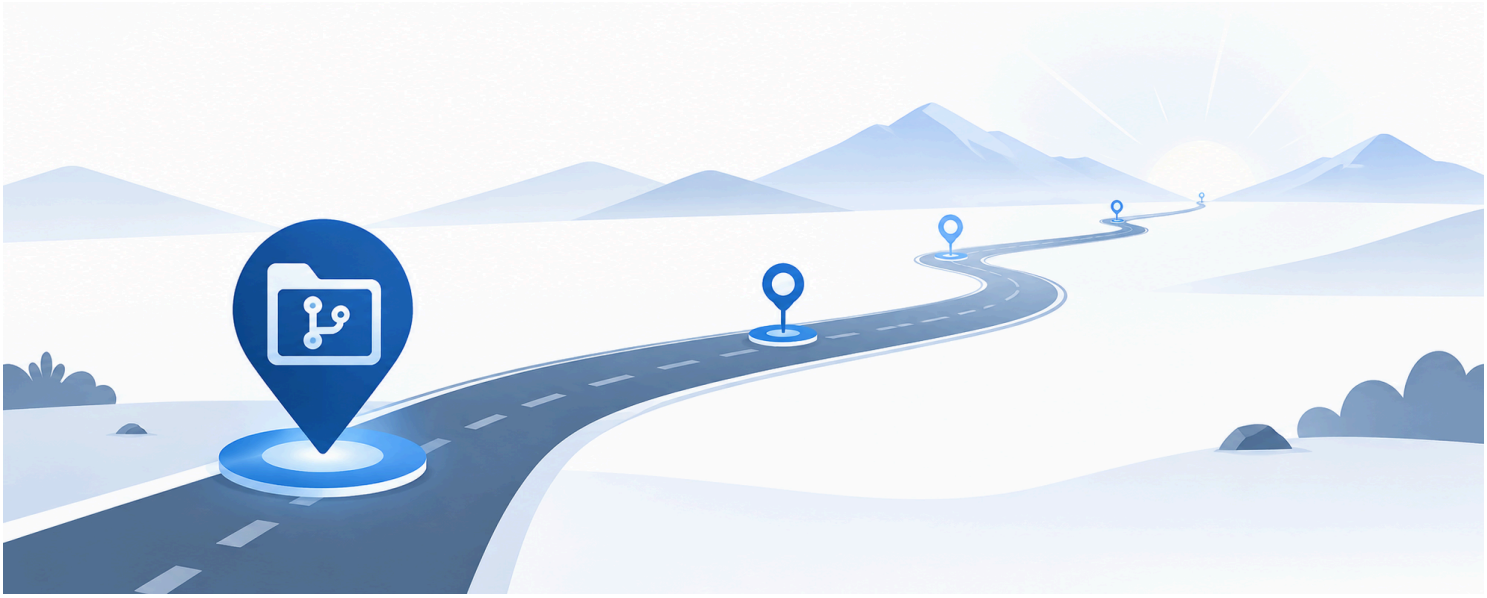
Ya viste, primero en conceptos y después en un recorrido completo narrado, cómo se arma un entorno de desarrollo de principio a fin.

Esta lección es el **laboratorio guiado** de esta clase: la parte donde dejas de observar y armas tú mismo el primer tramo de ese recorrido, con tu propio repositorio y tu propio primer commit.

---

## 🛠️ Qué vas a construir en este laboratorio

🎯 El **objetivo de esta actividad es concreto y acotado**: obtener la plantilla de repositorio base del programa, convertirla en tu propio repositorio Git, y registrar tu primer commit sobre esa estructura.



No vas a tocar todavía variables de entorno ni vas a hacer ninguna petición a una **API**: esa parte llega en el **Bloque 4**, con acompañamiento. Hoy el foco es exclusivamente que tu repositorio quede bien inicializado y que tu primer commit represente correctamente el punto de partida.

---

## 📦 Por qué se empieza desde la plantilla del programa

La plantilla de **repositorio base** no es un capricho de organización: define las convenciones que vas a reutilizar en cada proyecto de este programa, desde este **primer commit** hasta el repositorio que entregues en el **Gate 3**. Partir todos desde la misma base facilita que el equipo docente revise tu evidencia, y facilita que tú mismo reconozcas la estructura cuando trabajes en **proyectos posteriores del programa**.

La **plantilla** está adjunta directamente en esta lección de la plataforma, como un archivo comprimido. No hace falta clonar nada ni pasar por **GitHub** para obtenerla: descárgala directo desde aquí, para que el **repositorio** que quede inicializado sea tuyo desde el primer comando, no una copia del historial de otra persona.

**Plantilla Repositorio Base**

Descomprime esa carpeta en tu computador y ábrela en Visual Studio Code: esa carpeta, sin ningún historial de Git todavía, es tu punto de partida real para este laboratorio.

---

## ⚙️ Inicializar el repositorio y hacer tu primer commit, a mano

Con la carpeta ya abierta en VS Code, sigue este orden exacto, sin saltarte ningún paso.

➡️ **Inicializa el control de versiones** sobre la carpeta descomprimida.
***En VS Code**: panel de Source Control en la barra lateral izquierda, botón "Initialize Repository".*

```
git init
git branch -M main
```

**Por qué el segundo comando**: la instalación de **Git** en tu computador puede seguir creando la rama por defecto como **"master"** en vez de "main" (esto depende de cómo esté configurado tu **Git**). Si no renombras la rama ahora, el "**git push -u origin main**" de más adelante va a fallar con un error como "src refspec main does not match any".




➡️ **Revisa el estado antes de registrar nada**, para confirmar que los archivos que aparecen son los que esperabas, ni más ni menos.

***En VS Code**: el mismo panel de Source Control ya lista los archivos detectados bajo "Changes".*

```
git status
```

Así se ve, en términos generales, la salida esperada (el listado exacto de archivos depende de los que traiga tu plantilla):

```
On branch main
No commits yet
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        README.md
        ...
nothing added to commit but untracked files present (use "git add" to track)
```



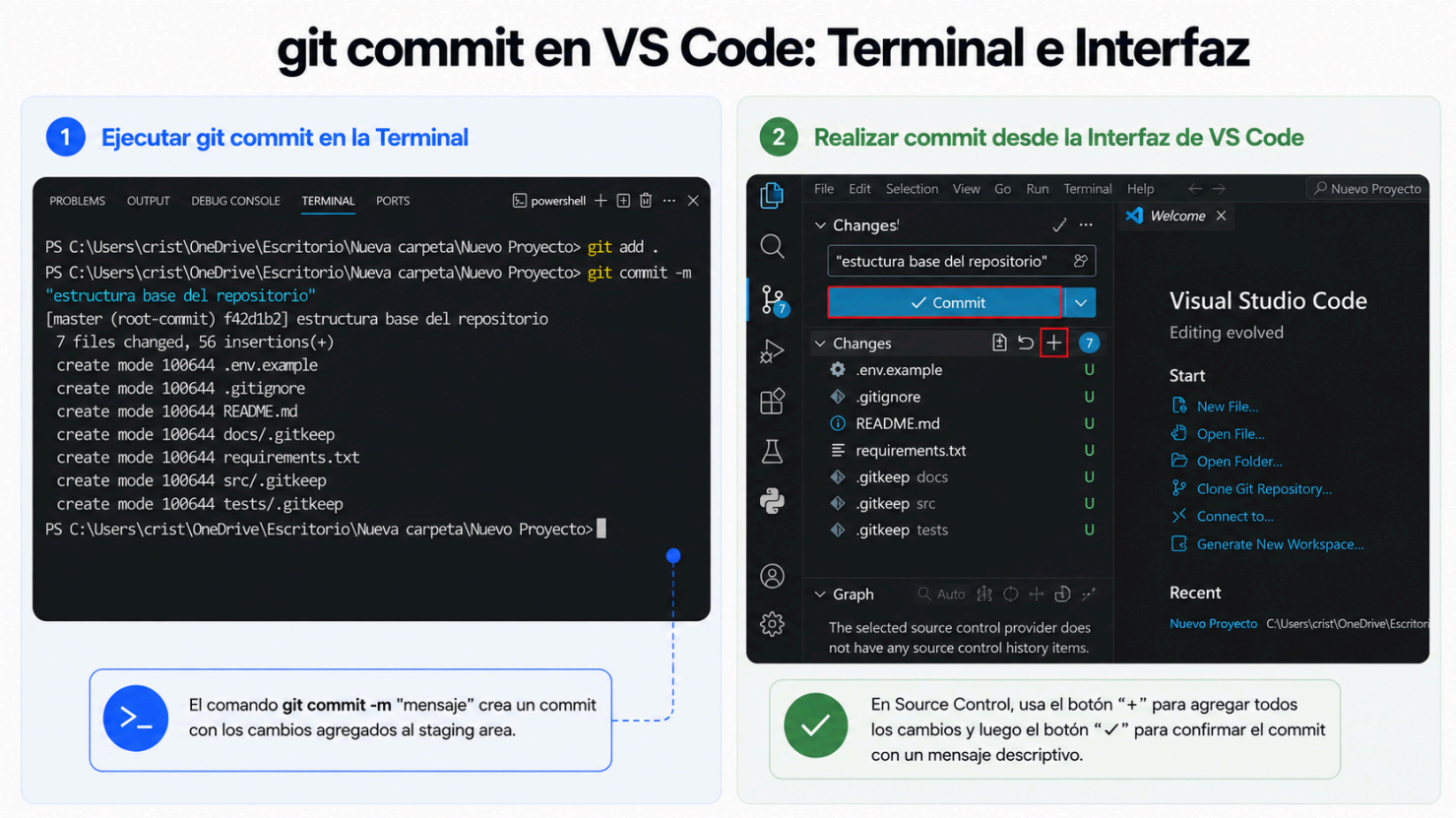

➡️ **Agrega los archivos y registra un único commit** con toda la estructura base tal como la descargaste, sin modificaciones propias todavía, y con un mensaje que describa exactamente eso (por ejemplo "estructura base del repositorio", no un mensaje genérico). Este primer commit, hecho por ti y no generado automáticamente por ninguna plataforma, es lo que da inicio formal al historial de tu proyecto en este programa.

***En VS Code**: en "Changes", clic en el ícono "+" para agregar todo, escribe el mensaje en el campo de texto de arriba, y presiona el botón de confirmar, el ícono de visto bueno.*

```
git add .
git commit -m "estructura base del repositorio"
```

Así se ve, en términos generales, la salida esperada (el hash del commit y la cantidad de archivos van a ser distintos en tu caso):

```
[main (root-commit) 1a2b3c4] estructura base del repositorio
N files changed, M insertions(+)
```




➡️ **Crea un repositorio vacío** en tu propia cuenta de **GitHub** (si todavía no tienes una cuenta, créala gratis en github.com antes de seguir), sin README ni licencia, para que quede completamente limpio. Si estás realizando este proceso mediante la **interfaz de Visual Studio Code**, puedes omitir este paso, ya que al inicializar y publicar el repositorio desde VS Code se creará automáticamente un nuevo repositorio en GitHub.

***En github.com**: botón "New repository", elige nombre y visibilidad, y no marques ninguna opción de inicialización (ni README, ni .gitignore, ni licencia).*




➡️ **Conéctalo como remoto de tu repositorio local**, y sube tu commit.

***En VS Code**: botón "Publish Branch" que aparece en la esquina inferior izquierda apenas haces tu primer commit. Luego, ingresa el nombre del nuevo repositorio y selecciona la opción para crearlo como público.*

```
# reemplaza la URL por la de tu propio repositorio, por ejemplo:
# git remote add origin https://github.com/tu-usuario/nombre-repositorio.git
git remote add origin URL-DE-TU-REPOSITORIO
git push -u origin main
```




✅ **Verifica dos cosas al terminar**:

- que el **repositorio quedó conectado** a ese remoto en tu cuenta de GitHub (todo el trabajo de este programa vive ahí, sin excepción),
- y que ese **primer commit efectivamente llegó al remoto** y es visible ahí, no solo en tu copia local.

---

## ⚠️ Errores frecuentes en este primer paso

Los tropiezos más comunes en este laboratorio son pocos y fáciles de anticipar:

📁

**Inicializar el repositorio un nivel de más o de menos**

A veces el descompresor crea una carpeta contenedora de más (una carpeta que a su vez contiene otra carpeta con los archivos reales). Confirma que estás parado exactamente en la carpeta que tiene el **README.md** y los demás archivos.



**Hacer el commit antes de revisar el estado**

Así se arrastran archivos que no correspondían, como carpetas temporales del sistema operativo.



**Dejar el commit solo en local**

Olvidar conectarlo con el remoto en **GitHub** deja la evidencia únicamente en tu propio computador.

Si te encuentras con cualquiera de estos tres casos, **corrígelo antes de seguir a la siguiente lección**: es más fácil arreglar el punto de partida ahora que más adelante, con más commits encima.

---

## 🧭 Qué sigue después de este laboratorio

Con tu repositorio inicializado y tu primer commit ya en GitHub, terminas el tramo de esta clase que te correspondía.

### 🧭 Siguiente lección

Bloque 3 · Construcción del entorno

En el siguiente bloque vas a replicar el resto del recorrido completo (entorno virtual, dependencias, guía de troubleshooting por sistema operativo) sobre este mismo repositorio, con acompañamiento entre pares.
Y en el **Bloque 4** vas a cerrar la unidad configurando variables de entorno y probando tu primera petición autenticada, momento en el que tu repositorio ya va a cumplir con el entregable completo de esta unidad y vas a ver cómo se entrega formalmente.

Última modificación: jueves, 24 de septiembre de 2026, 10:54